

Chapter 9: Lists

Fundamentals of Python: Data Structures, Second Edition

Learning Objectives

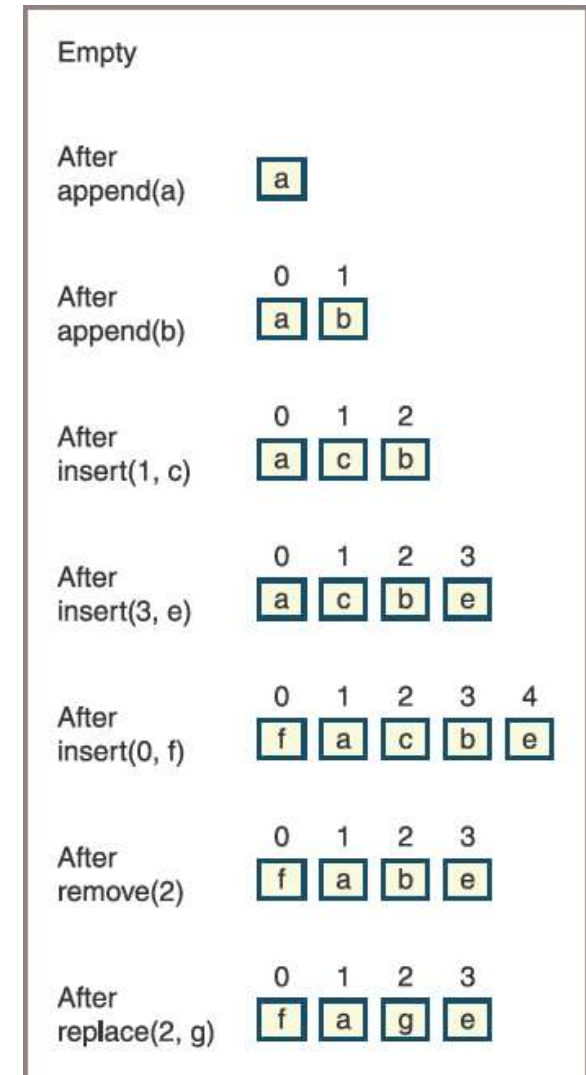
- Describe the features of a list and the operations on it
- Choose a list implementation based on its performance characteristics
- Recognize applications where it is appropriate to use a list
- Describe the features of a list iterator and the operations on it
- Recognize applications where it is appropriate to use a list iterator
- Develop some recursive list-processing functions

Overview of Lists (1 of 3)

- A list supports manipulation of items at any point within a linear collection
- Common examples of lists
 - A recipe – a list of instructions
 - A string – a list of characters
 - A document – a list of words
 - A file – a list of data blocks on a disk
- Array implementations of lists use physical positions to represent logical order:
 - Linked implementations do not

Overview of Lists (2 of 3)

- Figure 9.1 shows how a list changes in response to a succession of operations
- Figure 9-1: The states in the lifetime of a list



Overview of Lists (3 of 3)

- Table 9.1 The Operations Used in Figure 9.1

Operation	What It Does
<code>add(item)</code>	Adds <code>item</code> to the tail of the list.
<code>insert(index, item)</code>	Inserts <code>item</code> at position <code>index</code> , shifting other items to the right by one position if necessary.
<code>replace(index, item)</code>	Replaces the item at position <code>index</code> with <code>item</code> .
<code>pop(index)</code>	Removes the item at position <code>index</code> , shifting other items to the left by one position if necessary.

Using Lists

- Categories of operations
 - Index-based
 - Content-based
 - Position-based

Index-Based Operations (1 of 2)

- Index-based operations
 - Manipulate items at designated indices within a list
- In the case of array-based implementations,
 - These operations provide the convenience of random access
- Suppose a list contains n items:
 - Because a list is ordered linearly, you can refer unambiguously to an item in a list via its relative position from the head of the list using an index that runs from 0 to $n - 1$
 - Thus, the head is at index 0, and the tail is at index $n - 1$

Index-Based Operations (2 of 2)

- Table 9.2 Index-Based Operations for a List

List Method	What It Does
<code>L.Insert (I, item)</code>	Adds <code>item</code> at index <code>i</code> , after shifting items to the right by one position.
<code>L.Pop (i=None)</code>	Removes and returns the item at index <code>i</code> . If <code>i</code> is absent, removes and returns the last item.
<code>L [i]</code>	Returns the item at index <code>i</code> .
<code>L [i] = item</code>	Replaces the item at index <code>i</code> with <code>item</code> .

Content-Based Operations (1 of 2)

- Content-based operations
 - Based not on an index, but on the content of a list
 - Usually expect an item as an argument and do something with it and the list
- Table 9.3 lists three basic content-based operations for a list named **L**

Content-Based Operations (2 of 2)

- Table 9.3 Content-Based Operations for a List

List Method	What It Does
<code>L.add(item)</code>	Adds <code>item</code> after the list's tail.
<code>L.remove(item)</code>	Removes <code>item</code> from the list.
	<i>Precondition:</i> <code>item</code> is in the list.
<code>L.index(item)</code>	Returns the position of the first instance of <code>item</code> in the list.
	<i>Precondition:</i> <code>item</code> is in the list.

Position-Based Operations (1 of 14)

- Position-based operations
 - Performed relative to a currently established position called the cursor
 - Allow the programmer to navigate through a list by moving the cursor
 - A separate object called a *list iterator* provides these operations
- A list iterator supports movement to
 - Previous positions
 - Directly to the first position
 - Directly to the last position
- Also supports
 - Insertions, replacements, and removal of items at cursor positions

Position-Based Operations (2 of 14)

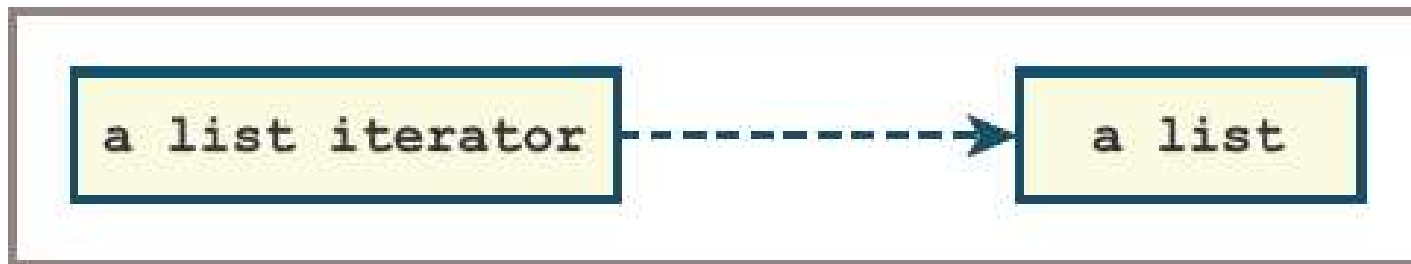
- A list iterator is created by running the `listIterator` method on a list:

```
listIterator=aList.listIterator ()
```

- The list can be either empty or already have items in it:
 - List is sometimes called the list iterator's *backing store*
- With respect to items in its backing store, a list iterator's cursor is always in one of three places:
 - Just before the first item
 - Between two adjacent items
 - Just after the last item

Position-Based Operations (3 of 14)

Figure 9-2: A list iterator attached to its backing store (a list)



Position-Based Operations (4 of 14)

- Table 9.4 Navigational Operations for a List Iterator

Operation	What It Does
<code>LI.hasNext()</code>	Returns True if there is an item after the cursor. Returns False if the cursor is undefined or is positioned after the last item.
<code>LI.next()</code>	Returns the next item and moves the cursor to the right by one position.
<code>LI.hasPrevious()</code>	Returns True if there is an item before the cursor. Returns False if the cursor is undefined or is positioned before the first item.
<code>LI.previous()</code>	Returns the previous item and moves the cursor to the left by one position.
<code>LI.first()</code>	Moves the cursor before the first item, if there is one.
<code>LI.last()</code>	Moves the cursor after the last item, if there is one.

Position-Based Operations (5 of 14)

- Table 9.5 Mutator Operations for a List Iterator

Operation	What It Does
<code>LI.insert(item)</code>	If the cursor is defined, inserts <code>item</code> after it; otherwise, inserts <code>item</code> at the tail of the list.
<code>LI.remove()</code>	Removes the item returned by the most recent <code>next</code> or <code>previous</code> operation.
<code>LI.replace(item)</code>	Replaces the item returned by the most recent call of <code>next</code> or <code>previous</code> operation.

Position-Based Operations (6 of 14)

- Table 9.6 The Effects of List Iterator Operations on a List

Operation	Position after Operation	The List after Operation	Value Returned	Comment
Instantiation	Undefined	Empty		A new list iterator.
<code>insert(a)</code>	Undefined	a		When the cursor is undefined, each item inserted goes at the tail of the list.
<code>insert(b)</code>	Undefined	a b		
<code>hasNext()</code>	Undefined		False	When the cursor is undefined, there is no next or previous item.
<code>first()</code>	0	, a b		Establish the cursor before the first item, if there is one.

Position-Based Operations (7 of 14)

- Table 9.6 The Effects of List Iterator Operations on a List (continued)

Operation	Position after Operation	The List after Operation	Value Returned	Comment
<code>hasNext()</code>	0		True	There is an item to the right of the cursor, so there is a next item.
<code>next()</code>	1	a , b	a	Return a and move the cursor to the right.
<code>replace(c)</code>	1	c, b		Replace a , the item most recently returned by <code>next</code> , with c .
<code>next()</code>	2	c b ,	b	Return b and move the cursor to the right.
<code>next()</code>	2	c b,	Exception	The cursor is at the tail of the list, so it is impossible to move to the next item.

Position-Based Operations (8 of 14)

- Table 9.6 The Effects of List Iterator Operations on a List (continued)

Operation	Position after Operation	The List after Operation	Value Returned	Comment
<code>hasNext()</code>	2	c b,	False	The cursor is at the tail of the list; therefore, there is no next item.
<code>hasPrevious()</code>	2	c b,	True	There is an item to the left of the cursor, so there is a previous item.
<code>previous()</code>	1	c b,	b	Return b and move the cursor to the left.
<code>insert(e)</code>	1	c, e b		Inserts e to the right of the cursor position.
<code>remove()</code>	1	c, e b	Exception	An insert has occurred since the most recent next or previous .

Position-Based Operations (9 of 14)

- Table 9.6 The Effects of List Iterator Operations on a List (continued)

Operation	Position after Operation	The List after Operation	Value Returned	Comment
<code>previous()</code>	0	,c e b	c	Return <code>c</code> and move the cursor to the left.
<code>remove()</code>	0	,e b		Remove <code>c</code> , the item most recently returned by <code>previous</code> .

Position-Based Operations (10 of 14)

- Code segment illustrating the use of a list iterator:

```
print("Create a list with 1-9")
lyst=ArrayList(range(1, 10))
print("Length:", len(lyst))
print("Items (first to last): ", lyst)
# Create and use a list iterator
listIterator=lyst.listIterator()
print("Forward traversal: ", end="")
listIterator.first()
while listIterator.hasNext():
    print(listIterator.next(), end=" ")
print("\nBackward traversal: ", end="")
listIterator.last()
while listIterator.hasPrevious():
    print(listIterator.previous(), end=" ")
print("\nInserting 10 before 3: ", end="")
listIterator.first()
for count in range(2):
    listIterator.next()
```

Position-Based Operations (11 of 14)

- Code segment illustrating the use of a list iterator

(continued):

```
listIterator.insert(10)
print(lyst)
print("Removing 2: ", end=" ")
listIterator.first()
for count in range(3):
    listIterator.next()
listIterator.remove()
print(lyst)
print("Removing all items")
listIterator.first()
while listIterator.hasNext():
    listIterator.next()
    listIterator.remove()
print("Length: ", len(lyst))
```

Position-Based Operations (12 of 14)

- Output of the code segment:

Create a list with 1-9

Length: 9

Items (first to last): 1 2 3 4 5 6 7 8 9

Forward traversal: 1 2 3 4 5 6 7 8 9

Backward traversal: 9 8 7 6 5 4 3 2 1

Inserting 10 before 3: 1 2 10 3 4 5 6 7 8 9

Removing 2: 1 10 3 4 5 6 7 8 9

Removing all items

Length: 0

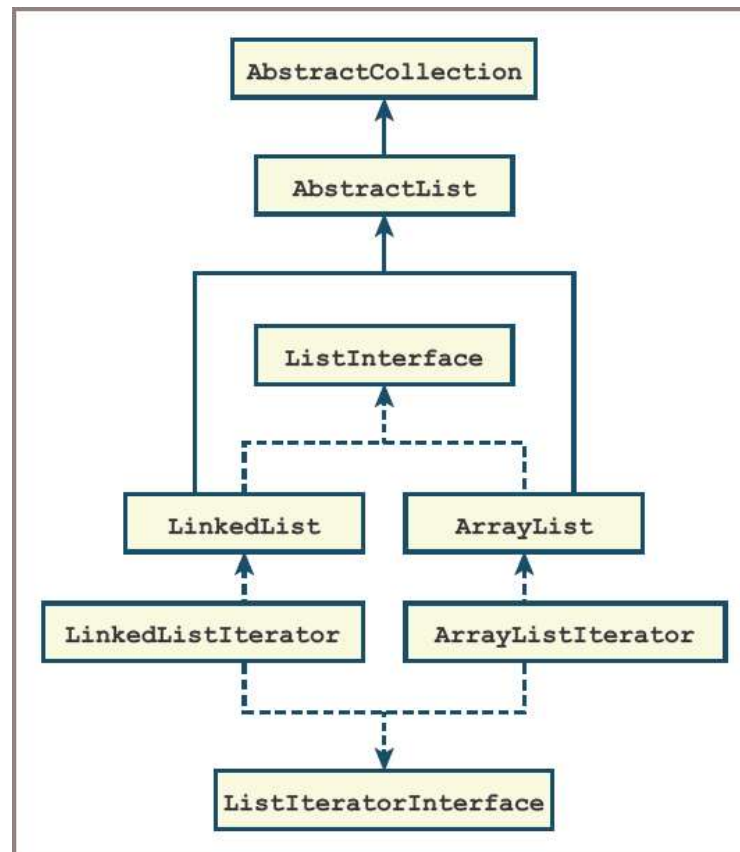
Position-Based Operations (13 of 14)

- Table 9.7 Summary of Basic List Operations

Operation	Content-Based Operation	Position-Based Operation
<code>L.insert(i, item)</code>	<code>L.add(item)</code>	<code>LI.hasNext()</code>
<code>L.pop(i)</code>	<code>L.remove(item)</code>	<code>LI.next()</code>
<code>L[i]</code>	<code>L.index(item)</code>	<code>LI.hasPrevious()</code>
<code>L[i] = item</code>		<code>LI.first()</code> <code>LI.last()</code> <code>LI.insert(item)</code> <code>LI.remove(item)</code> <code>LI.replace(item)</code>

Position-Based Operations (14 of 14)

Figure 9-3: The interfaces and implementing classes for lists and list iterators



Applications of Lists

- This section examines three important applications:
 - Heap-storage management
 - Disk file management
 - Implementation of other collections

Heap-Storage Management

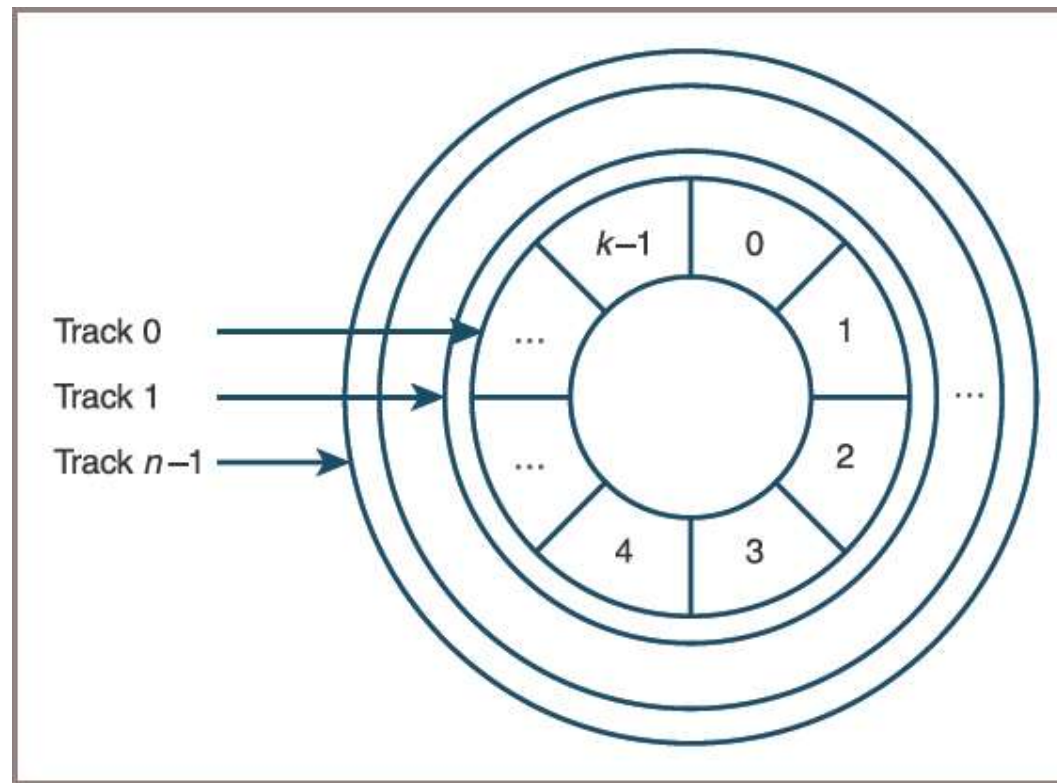
- The object heap is an area of memory from which the Python virtual machine allocates segments of various sizes for all new data objects
- When an object no longer can be referenced from a program, the Python Virtual Machine (PVM) can return that object's memory segment to the heap for use by other objects
- Heap-management schemes can have a significant impact on an application's overall performance:
 - Contiguous blocks of free space on the heap are linked in a free list
 - When an application instantiates a new object, the PVM searches the free list for the first block large enough to hold the object
 - When the object is no longer needed, the garbage collector returns the object's space to the free list

Organization of Files on a Disk (1 of 3)

- A computer's file system has three major components:
 - A directory of files
 - The files
 - Free space
- Disk's surface is divided into concentric tracks
 - Each track is further subdivided into sectors
 - Number of tracks vary depending on the disk's capacity and physical size
 - All tracks contain the same number of sectors and all sectors contain the same number of bytes

Organization of Files on a Disk (2 of 3)

Figure 9-4: Tracks and sectors on the surface of a disk



Organization of Files on a Disk (3 of 3)

- A file system's directory is organized as a hierarchical collection
- A file system does not experience the same fragmentation problem encountered in Python's object heap:
 - There is still a difficulty
 - To transfer data to or from the disk, read/write heads must first be positioned to the correct track:
 - The disk must rotate until the desired sector is under the heads
- Data can be transferred to or from several adjacent sectors during a single rotation:
 - A disk system's performance is optimized when multisector files are not scattered across the disk

Implementation of Other Collections

- Two ways in which lists are used to implement other collections
 - Extend the list class, making the new class a subclass of the list class
 - Use an instance of the list class within the new class and let the list contain the data items
- Collections that use lists also inherit their performance characteristics
- Primary advantage
 - Coding becomes easier
 - Instead of operating on a concrete array or linked structure, the implementer of a stack needs only call the appropriate list methods

List Implementations

- This section develops array-based and linked implementations

The Role of the **AbstractList** Class (1 of 3)

- Each concrete list class is subclassed under an abstract class
 - Called **AbstractList**
- Because this class is a subclass of **AbstractCollection**
 - The list classes inherit the usual collection methods, as well as the variable **self.size**
- You can also include some other list methods in **AbstractList** to reduce the number of methods you need to define in the concrete classes

The Role of the AbstractList Class (2 of 3)

- Code for the **AbstractList** class:

```
"""
File: abstractlist.py
Author: Ken Lambert
"""

from abstractcollection import AbstractCollection

class AbstractList(AbstractCollection):
    """An abstract list implementation."""

    def __init__(self, sourceCollection):
        """Maintains a count of modifications to the list."""
        self.modCount = 0
        AbstractCollection.__init__(self, sourceCollection)

    def getModCount(self):
        """Returns the count of modifications to the list."""
        return self.modCount

    def incModCount(self):
        """Increments the count of modifications
        to the list."""
        self.modCount += 1
```

The Role of the AbstractList Class (3 of 3)

- Code for the **AbstractList** class (continued):

```
def index(self, item):
    """Precondition: item is in the list.
    Returns the position of item.
    Raises: ValueError if the item is not in the list."""
    position = 0
    for data in self:
        if data == item:
            return position
        else:
            position += 1
    if position == len(self):
        raise ValueError(str(item) + " not in list.")

def add(self, item):
    """Adds the item to the end of the list."""
    self.insert(len(self), item)

def remove(self, item):
    """Precondition: item is in self.
    Raises: ValueError if item is not in self.
    Postcondition: item is removed from self."""
    position = self.index(item)
    self.pop(position)
```

An Array-Based Implementation (1 of 4)

- The array-based implementation of the list interface is a class called **ArrayList**:
 - An **ArrayList** maintains its data items in an instance of the **Array** class
 - Has an initial default capacity that is automatically increased when necessary
- Because the **ArrayList** class is a subclass of **AbstractList**
 - Only the index-based methods, the `__iter__` method, and the **listIterator** methods are defined here
 - The index-based operations `__getitem__` and `__setitem__` simply use the subscript operator on the array variable **self.items**

An Array-Based Implementation (2 of 4)

- Code for the class **ArrayList**:

```
"""
File: arraylist.py
Author: Ken Lambert
"""

from arrays import Array
from abstractlist import AbstractList
from arraylistiterator import ArrayListIterator
class ArrayList(AbstractList):
    """An array-based list implementation."""

    DEFAULT_CAPACITY = 10

    def __init__(self, sourceCollection=None):
        """Sets the initial state of self, which includes the
        contents of sourceCollection, if it's present."""
        self.items = Array(ArrayList.DEFAULT_CAPACITY)
        AbstractList.__init__(self, sourceCollection)

    # Accessor methods
    def __iter__(self):
        """Supports iteration over a view of self."""
        cursor = 0
        while cursor < len(self):
            yield self.items[cursor]
            cursor += 1

    def __getitem__(self, i):
        """Precondition: 0 <= i < len(self)
        Returns the item at position i.
        Raises: IndexError."""
        if i < 0 or i >= len(self):
            raise IndexError("List index out of range")
        return self.items[i]
```

An Array-Based Implementation (3 of 4)

- Code for the class `ArrayList` (continued):

```
# Mutator methods
def __setitem__(self, i, item):
    """Precondition: 0 <= i < len(self)
    Replaces the item at position i.
    Raises: IndexError."""
    if i < 0 or i >= len(self):
        raise IndexError("List index out of range")
    self.items[i] = item

def insert(self, i, item):
    """Inserts the item at position i."""
    # Resize array here if necessary
    if i < 0: i = 0
    elif i > len(self): i = len(self)
    if i < len(self):
        for j in range(len(self), i, -1):
            self.items[j] = self.items[j - 1]
    self.items[i] = item
    self.size += 1
    self.incModCount()
```

An Array-Based Implementation (4 of 4)

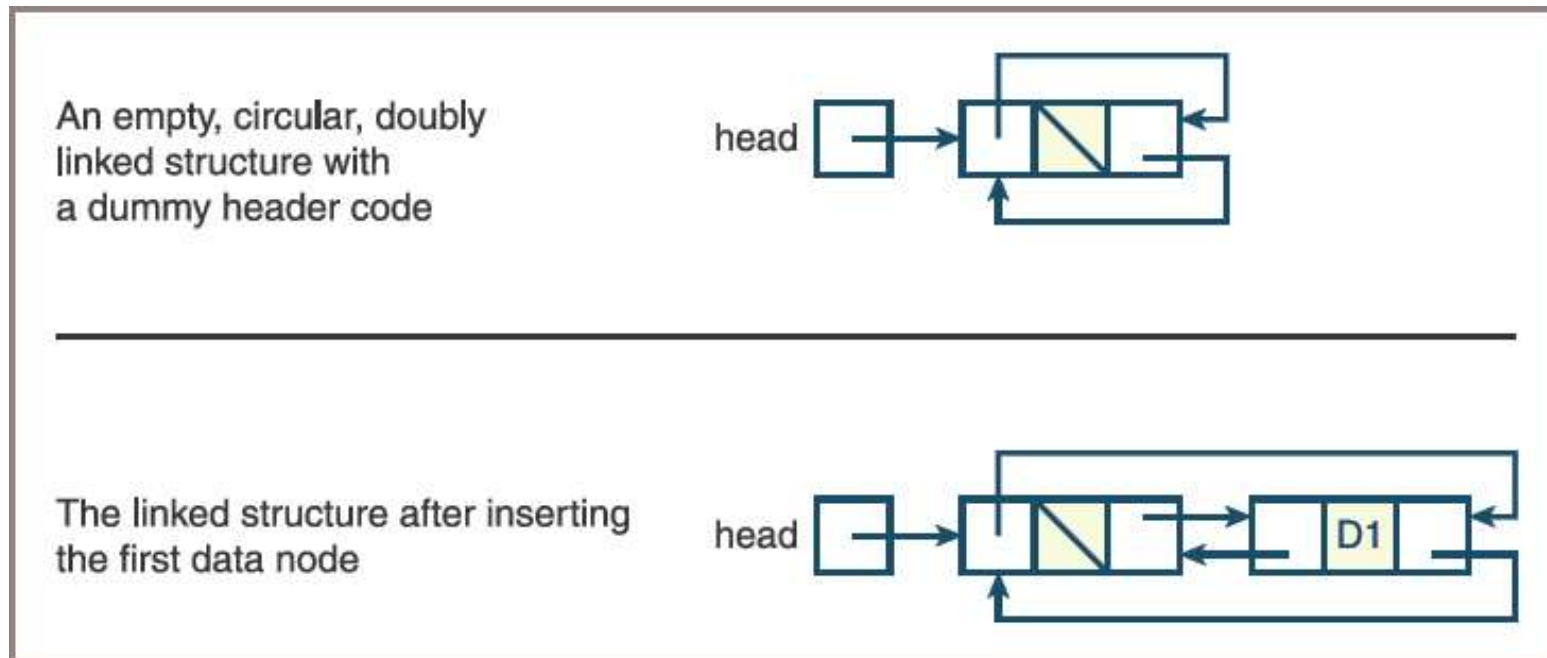
- Code for the class `ArrayList` (continued):

```
def pop(self, i = None):
    """Precondition: 0 <= i < len(self).
    Removes and returns the item at position i.
    If i is None, i is given a default of len(self) - 1.
    Raises: IndexError."""
    if i == None: i = len(self) - 1
    if i < 0 or i >= len(self):
        raise IndexError("List index out of range")
    item = self.items[i]
    for j in range(i, len(self) - 1):
        self.items[j] = self.items[j + 1]
    self.size -= 1
    self.incModCount()
    # Resize array here if necessary
    return item

def listIterator(self):
    """Returns a list iterator."""
    return ArrayListIterator(self)
```

A Linked Implementation (1 of 4)

Figure 9-5: Two circular, doubly linked structures with sentinel nodes



A Linked Implementation (2 of 4)

- Setup code for the `LinkedList` class:

```
"""
File: linkedlist.py
Author: Ken Lambert
"""

from node import TwoWayNode
from abstractlist import AbstractList

class LinkedList(AbstractList):
    """A link-based list implementation."""

    def __init__(self, sourceCollection=None):
        """Sets the initial state of self, which includes the
        contents of sourceCollection, if it's present."""
        # Uses a circular structure with a sentinel node
        self.head = TwoWayNode()
        self.head.previous = self.head.next = self.head
        AbstractList.__init__(self, sourceCollection)

    # Accessor methods
    def __iter__(self):
        """Supports iteration over a view of self."""
        cursor = self.head.next
        while cursor != self.head:
            yield cursor.data
        cursor = cursor.next
```


A Linked Implementation (3 of 4)

- Code for `getNode`, `_setitem_`, and `insert` methods:

```
# Helper method returns node at position i
def getNode(self, i):
    """Helper method: returns a pointer to the node
    at position i."""
    if i == len(self):      # Constant-time access to head node
        return self.head
    if i == len(self) - 1:  # or last data node
        return self.head.previous
    probe = self.head.next
    while i > 0:
        probe = probe.next
        i -= 1
    return probe
```

A Linked Implementation (4 of 4)

- Code for `getNode`, `_setitem_`, and `insert` methods (continued):

```
# Mutator methods
def _setitem_(self, i, item):
    """Precondition: 0 <= i < len(self)
    Replaces the item at position i.
    Raises: IndexError."""
    if i < 0 or i >= len(self):
        raise IndexError("List index out of range")
    self.getNode(i).data = item

def insert(self, i, item):
    """Inserts the item at position i."""
    if i < 0: i = 0
    elif i > len(self): i = len(self)
    theNode = self.getNode(i)
    newNode = TwoWayNode(item, theNode.previous, theNode)
    theNode.previous.next = newNode
    theNode.previous = newNode
    self.size += 1
    self.incModCount()
```

Time and Space Analysis for the Two Implementations (1 of 3)

- Table 9.8 Average Running Times for List Operations

List Method	ArrayList	LinkedList
<code>__getitem__(i)</code>	$O(1)$	$O(n)$
<code>__setitem__(i, item)</code>	$O(1)$	$O(n)$
<code>insert(i, item)</code>	$O(n)$	$O(n)$
<code>pop(i)</code>	$O(n)$	$O(n)$
<code>add(item)</code>	$O(1)$	$O(1)$
<code>remove(item)</code>	$O(n)$	$O(n)$
<code>index(item)</code>	$O(n)$	$O(n)$

Time and Space Analysis for the Two Implementations (2 of 3)

- A minimal array implementation requires memory for the following items:
 - An array that can hold **capacity** references, where **capacity** $\geq n$
 - A reference to the array
 - Variables for the number of items and the mod count
- Thus, the total space requirement for the minimal array implementation is **capacity + 3**

Time and Space Analysis for the Two Implementations (3 of 3)

- The linked implementation requires memory for the following items:
 - $n + 1$ nodes, where each node contains three references
 - A reference to the head node
 - Variables for the number of items and the mod count
- Thus, the total space requirement for the linked implementation is $3n + 6$

Implementing a List Iterator

- In this section, you develop a list iterator for an array-based list

Role and Responsibilities of a List Iterator

- Three types of preconditions
 - A programmer cannot run a **next** or a **previous** operation if the **hasNext** or the **hasPrevious** operation returns **False**, respectively
 - A programmer cannot run consecutive mutator methods on a list iterator:
 - A **next** or a **previous** must first be run before each mutation to establish a cursor position
 - A programmer cannot run mutations on the list itself, with the list's mutator methods, while using a list iterator on that list
- To help determine some of these preconditions, the list iterator maintains two additional variables:
 - The first one is its own mod count variable
 - The second variable tracks the position at which the list iterator can perform a mutation on the list

Setting Up and Instantiating a List Iterator Class (1 of 2)

- The list iterator class for array lists is called **ArrayListIterator**
- This class includes the following instance variables:
 - **self.backingStore** —The list on which the iterator is opened
 - **self.modCount** —The iterator's notion of the mod count
 - **self.cursor** —The cursor position manipulated by the iterator's navigational methods **first**, **last**, **hasNext**, **next**, **hasPrevious**, and **previous**
 - **self.lastItemPos** —The cursor position used by the iterator's mutator methods **insert**, **remove**, and **replace**
 - This position is established by running **next** or **previous** and is undefined after running **insert** or **remove**

Setting Up and Instantiating a List Iterator Class (2 of 2)

- Code for this portion of the `ArrayListIterator` class:

```
"""
File: arraylistiterator.py
Author: Ken Lambert
"""

class ArrayListIterator(object):
    """Represents the list iterator for an array list."""

    def __init__(self, backingStore):
        """Set the initial state of the list iterator."""
        self.backingStore = backingStore
        self.modCount = backingStore.getModCount()
        self.first()

    def first(self):
        """Resets the cursor to the beginning
        of the backing store."""
        self.cursor = 0
self.lastItemPos = -1
```

The Navigational Methods in the List Iterator (1 of 4)

- The navigational methods **hasNext** and **next** work with a cursor that is moving from the beginning of an array-based list to its end
- The method **hasNext** returns **True** as long as the cursor is less than the backing store's length
- The method **next** must check two preconditions before advancing the cursor and returning an item from the backing store:
 - First, the method **hasNext** must return **True**
 - Second, the two mod counts—one belonging to the list iterator and the other to the backing store—must be equal

The Navigational Methods in the List Iterator (2 of 4)

- Code for these two methods:

```
def hasNext(self):  
    """Returns True if the iterator has  
    a next item or False otherwise."""  
    return self.cursor < len(self.backingStore)  
  
def next(self):  
    """Preconditions: hasNext returns True.  
    The list has not been modified except by this  
    iterator's mutators.  
    Returns the current item and advances the cursor.  
    to the next item."""  
    if not self.hasNext():  
        raise ValueError("No next item in list iterator")  
    if self.modCount != self.backingStore.getModCount():  
        raise AttributeError(  
            "Illegal modification of backing store")  
    self.lastItemPos = self.cursor  
    self.cursor += 1  
    return self.backingStore[self.lastItemPos]
```

The Navigational Methods in the List Iterator (3 of 4)

- The methods **last**, **hasPrevious**, and **previous** work with a cursor that is moving from the end of an array-based list to its beginning
- The method **last** sets the position of the cursor to the right of the last item in the list
 - This position will be equal to the length of the list
- The method **hasPrevious** returns **True** if the cursor is greater than position 0
- The method **previous** checks the same two preconditions as the method **next**
 - It then decrements the cursor by 1, sets **self.lastItemPos** to **self.cursor**, and returns the item at **self.lastItemPos** in the backing store

The Navigational Methods in the List Iterator (4 of 4)

- Code for the three methods:

```
def last(self):
    """Moves the cursor to the end of the backing store."""
    self.cursor = len(self.backingStore)
    self.lastItemPos = -1

def hasPrevious(self):
    """Returns True if the iterator has a
    previous item or False otherwise."""
    return self.cursor > 0

def previous(self):
    """Preconditions: hasPrevious returns True.
    The list has not been modified except
    by this iterator's mutators.
    Returns the current item and moves
    the cursor to the previous item."""
    if not self.hasPrevious():
        raise ValueError("No previous item in list iterator")
    if self.modCount != self.backingStore.getModCount():
        raise AttributeError(
            "Illegal modification of backing store")
    self.cursor -= 1
    self.lastItemPos = self.cursor
    return self.backingStore[self.lastItemPos]
```

Mutator Methods in the List Iterator (1 of 3)

- The mutator methods **remove** and **replace** must check two preconditions
 - First, the cursor must be established, meaning that the variable **self.lastItemPos** must not equal **-1**
 - Second, the two mod counts must be equal
- The method **insert** checks only the precondition on the mod counts

Mutator Methods in the List Iterator (2 of 3)

- Code for the three mutator methods:

```
def replace(self, item):
    """Preconditions: the current position is defined.
    The list has not been modified except by this
    iterator's mutators."""
    if self.lastItemPos == -1:
        raise AttributeError(
            "The current position is undefined.")
    if self.modCount != self.backingStore.getModCount():
        raise AttributeError(
            "List has been modified illegally.")
    self.backingStore[self.lastItemPos] = item
    self.lastItemPos = -1

def insert(self, item):
    """Preconditions:
    The list has not been modified except by this
    iterator's mutators."""
    if self.modCount != self.backingStore.getModCount():
        raise AttributeError(
            "List has been modified illegally.")
    if self.lastItemPos == -1:
        # Cursor not defined, so add item to end of list
        self.backingStore.add(item)
    else:
        self.backingStore.insert(self.lastItemPos, item)
    self.lastItemPos = -1
    self.modCount += 1
```

Mutator Methods in the List Iterator (3 of 3)

- Code for the three mutator methods (continued):

```
def remove(self):
    """Preconditions: the current position is defined.
    The list has not been modified except by this
    iterator's mutators."""
    if self.lastItemPos == -1:
        raise AttributeError(
            "The current position is undefined.")
    if self.modCount != self.backingStore.getModCount():
        raise AttributeError(
            "List has been modified illegally.")
    item = self.backingStore.pop(self.lastItemPos)
    # If the item removed was obtained via next,
    # move cursor back
    if self.lastItemPos < self.cursor:
        self.cursor -= 1
    self.modCount += 1
    self.lastItemPos = -1
```


Design of a List Iterator for a Linked List

- It is possible to use the **ListIterator** class just described with a linked list
- However, because this implementation of the list iterator runs the index-based methods **__getitem__** and **__setitem__** on the backing store,
 - The **next**, **previous**, and **replace** operations will run in linear time when this backing store is a linked list

Time and Space Analysis of List Iterator Implementations

- The running times of all the methods in the linked implementation of a list iterator are $O(1)$
- Compare to the array-based implementation, whose **insert** and **remove** methods are both $O(n)$

Recursive List Processing

- *Lisp*
 - Stands for list processing
 - The list is the basic data structure of Lisp
- A Lisp list is a recursive data structure
 - Lisp programs often consist of a set of recursive functions for processing lists

Basic Operations on a Lisp-Like List (1 of 3)

- Users of a Lisp-like list have three basic functions to examine lists:
 - First function is a predicate named **isEmpty**:
 - This function returns **True** if its argument is an empty list, or **False** otherwise
 - The function **first** returns the data item at the head of the list
 - The function **rest** returns a list containing the data items after this first one.

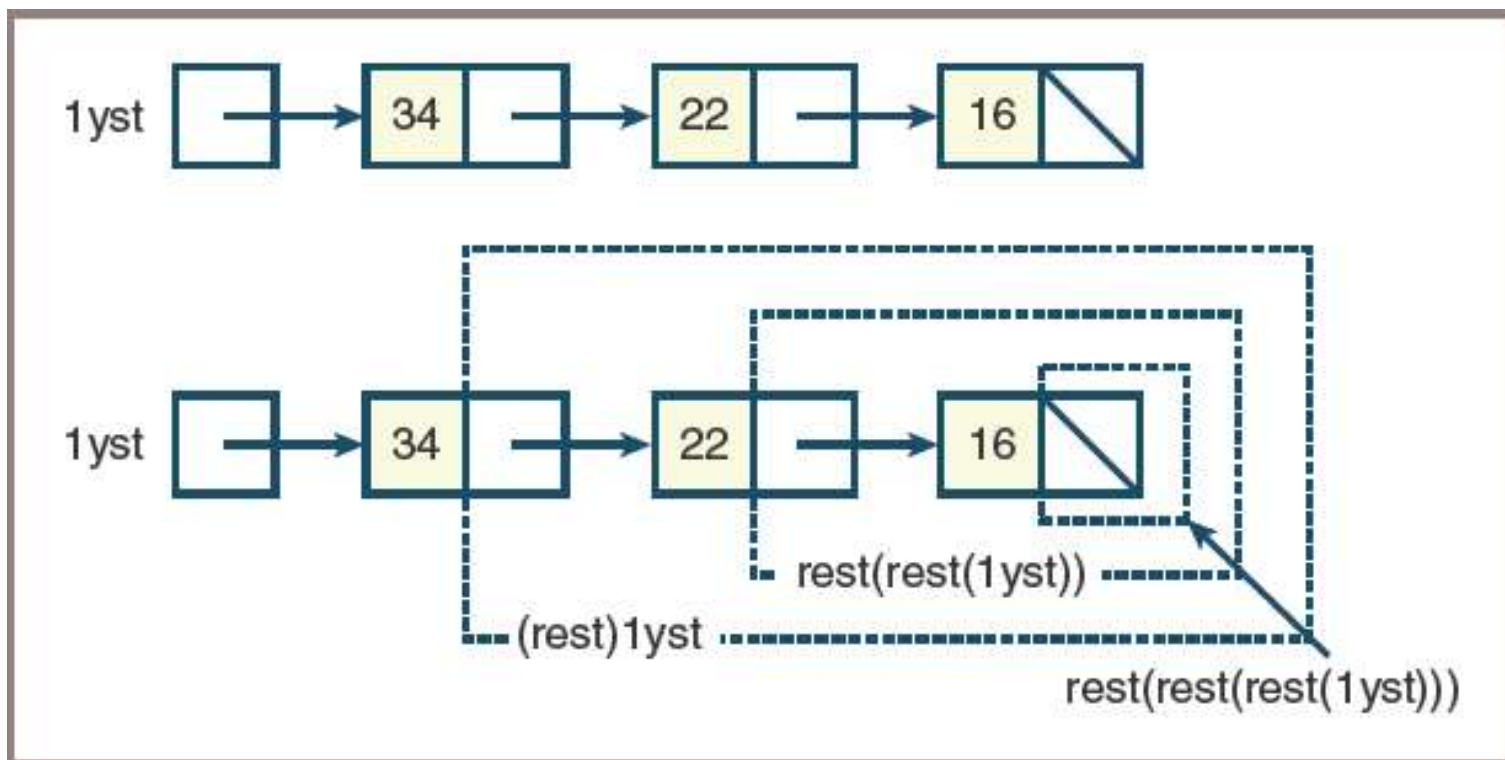
Basic Operations on a Lisp-Like List (2 of 3)

- Table 9.11 Applying the basic list functions to a list containing 34, 22, and 16

Function Application	Result
<code>isEmpty(lyst)</code>	Returns False .
<code>first(lyst)</code>	Returns 34.
<code>rest(lyst)</code>	Returns a list containing 22 and 16.
<code>first(rest(lyst))</code>	Returns 22.
<code>first(rest(rest(lyst)))</code>	Returns 16.
<code>isEmpty(rest(rest(rest(lyst))))</code>	Returns True .
<code>first(rest(rest(rest(lyst))))</code>	Raises an error (no data in an empty list).

Basic Operations on a Lisp-Like List (3 of 3)

Figure 9-10: A lisp-like list containing 34, 22, and 16



Recursive Traversals of a Lisp-Like List (1 of 4)

- The function **contains** searches a list for a given item:
 - This function expects a target item and a list as arguments and returns **True** or **False**
 - If the list is empty, you've run out of items to examine, so the function returns **False**
 - Otherwise, if the target item equals the first item in the list, the function returns **True**
- Otherwise, you use **contains** to search the rest of the list recursively for the given item

Recursive Traversals of a Lisp-Like List (2 of 4)

- The code:

```
def contains(item, lyst):  
    """Returns True if item is in lyst or  
    False otherwise."""  
    if isEmpty(lyst):  
        return False  
    elif item == first(lyst):  
        return True  
    else:  
        return contains(item, rest(lyst))
```


Recursive Traversals of a Lisp-Like List (3 of 4)

- The index-based function **get** returns the *i*th element of a given list
- The definition of **get** follows:

```
def get(index, lyst):  
    """Returns the item at position index in lyst.  
    Precondition: 0 <= index < length(lyst)"""  
    if index == 0:  
        return first(lyst)  
    else:  
        return get(index - 1, rest(lyst))
```

Recursive Traversals of a Lisp-Like List (4 of 4)

- Suppose you do not know the length of a list,
 - Which is a part of the precondition of the **get** function
- The definition of the length of a Lisp-like list can be stated recursively:
 - Its length is 0 if the list is empty
 - Otherwise, a list's length is one plus the length of the rest of the list after the first item
- Code for the recursive function **length**:

```
def length(lyst):  
    """Returns the number of items in lyst."""  
    if isEmpty(lyst):  
        return 0  
    else:  
        return 1 + length(rest(lyst))
```

Building a Lisp-Like List (1 of 5)

- A Lisp-like list has a single basic constructor function named **cons**:
 - This function expects two arguments: a data item and another list
- The relationships between the functions **cons**, **first**, and **rest** can be expressed algebraically in the following pair of equations:

```
first(cons(A, B)) == A  
rest(cons(A, B)) == B
```

Building a Lisp-Like List (2 of 5)

- Table 9.12 Building Lists with **cons**

Function Application or Variable Reference	Resulting List
<code>THE_EMPTY_LIST</code>	An empty list.
<code>cons(22, THE_EMPTY_LIST)</code>	A list containing 22.
<code>cons(11, cons(22, THE_EMPTY_LIST))</code>	A list containing 11 and 22.

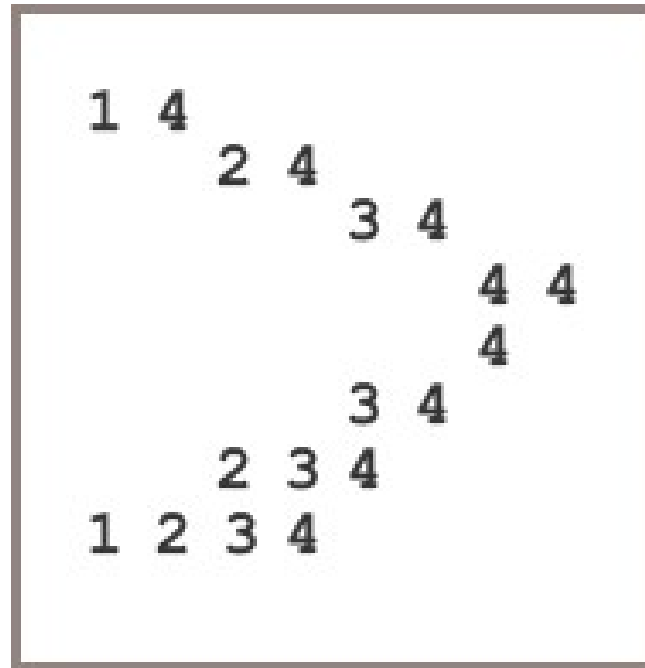
Building a Lisp-Like List (3 of 5)

- Code for `buildRange`:

```
def buildRange(lower, upper):  
    """Returns a list containing the numbers from  
    lower through upper.  
    Precondition: lower <= upper"""  
    if lower == upper:  
        return cons(lower, THE_EMPTY_LIST)  
    else:  
        return cons(lower, buildRange(lower + 1, upper))
```

Building a Lisp-Like List (4 of 5)

Figure 9-11: Tracing the recursive building of a list with buildRange



Building a Lisp-Like List (5 of 5)

- The **remove** function decrements the index and moves to the rest of the list on each recursive call
- Here is the code:

```
def remove(index, lyst):  
    """Returns a list with the item at index removed.  
    Precondition: 0 <= index < length(lyst)"""  
    if index == 0:  
        return rest(lyst)  
    else:  
        return cons(first(lyst),  
                     remove(index - 1, rest(lyst)))
```

The Internal Structure of a Lisp-Like List (1 of 2)

- A Lisp-like list consists of a sequence of nodes, where each node contains a data item named **data** and a link to the next node named **next**:
 - The **next** link in the last node is **None**
- A Lisp-like list is an abstract data type (ADT)
 - Includes four basic functions and the constant for the empty list
- The user of this ADT doesn't have to know anything about nodes, links, or pointers

The Internal Structure of a Lisp-Like List (2 of 2)

- Code:

```
"""
File: lisplist.py
Data and basic operations for Lisp-like lists.
"""

from node import Node

THE_EMPTY_LIST = None

def isEmpty(lyst):
    """Returns True if lyst is empty or False otherwise."""
    return lyst is THE_EMPTY_LIST

def first(lyst):
    """Returns the item at the head of lyst.
    Precondition: lyst is not empty."""
    return lyst.data

def rest(lyst):
    """Returns a list of the items after the head of lyst.
    Precondition: lyst is not empty."""
    return lyst.next

def cons(data, lyst):
    """Returns a list whose head is item and whose tail
    is lyst."""
    return Node(data, lyst)
```

Printing Lisp-Like Lists in IDLE with

`__repr__` (1 of 2)

- Add a method named `__repr__` to the **Node** class:
 - When Python evaluates a value for display in the IDLE shell, it looks for this method in the value's class
- If `__repr__` is included in the **Node** class,
 - You can build and return a string with the data of the entire sequence of nodes enclosed in parentheses

Printing Lisp-Like Lists in IDLE with

`__repr__` (2 of 2)

- Code:

```
class Node(object):
    """Represents a singly linked node."""

    def __init__(self, data, next = None):
        self.data = data
        self.next = next

    def __repr__(self):
        """Returns the string representation of a
        nonempty lisp list."""
        def buildString(lyst):
            if isEmpty(rest(lyst)):
                return str(first(lyst))
            else:
                return str(first(lyst)) + " " +
buildString(rest(lyst))
        return "(" + buildString(self) + ")"
```

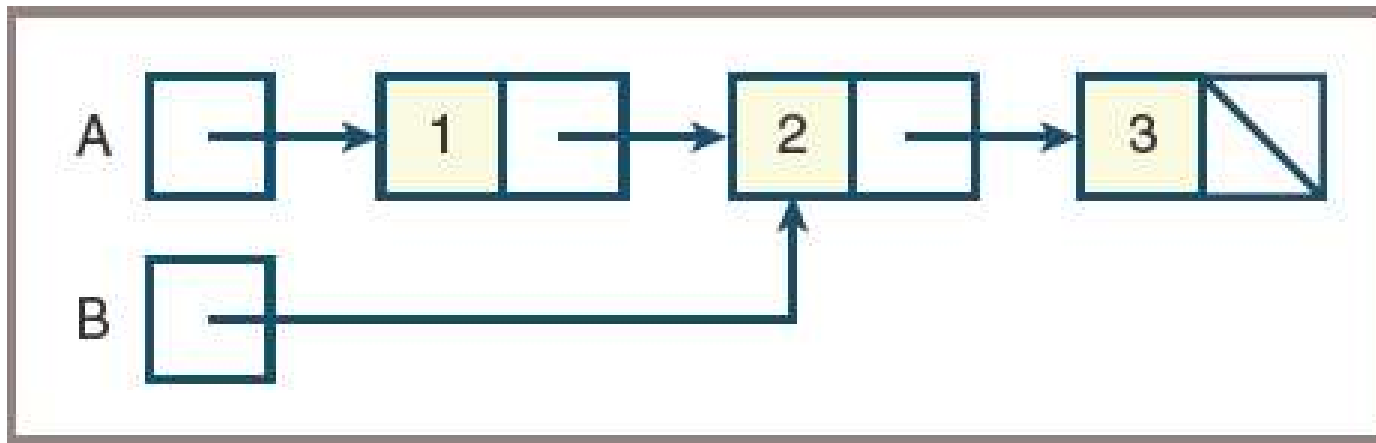
Lists and Functional Programming (1 of 2)

- The **remove** function does not change the structure of its list argument,
 - It simply returns a list with the *i*th item removed
- This code segment illustrates this by removing list A's first item and assigning the result to list B:

```
>>> A = buildRange(1, 3)
>>> print(A)
(1 2 3)
>>> B = remove(0, A)           # Remove the item at position 0
>>> print(B)
(2 3)
>>> print(A)                  # List referenced by A not changed
(1 2 3)
```

Lists and Functional Programming (2 of 2)

Figure 9-12: The shared structure of two lists A and B



Chapter Summary (1 of 2)

- A list is a linear collection that allows users to insert, remove, access, and replace elements at any position
- Operations on lists are index based, content based, or position based:
 - An index-based operation allows access to an element at a specified integer index
 - A position-based list lets the user scroll through it by moving a cursor
- List implementations are based on arrays or on linked structures:
 - A doubly linked structure is more convenient and faster when used with a list iterator than a singly linked structure

Chapter Summary (2 of 2)

- A sorted list is a list whose elements are always in ascending or descending order
- A list can have a recursive definition — it is either empty or consists of a data item and another list
 - The recursive structure of such lists supports a wide array of recursive list-processing functions